Laboratorium 7: Dziedziczenie i polimorfizm w języku Java

Dominik Delimata

Informatyka

1 rok

Laboratorium nr 1

ZADANIA

Zadanie 1. Zaimplementuj klasę Osoba. Klasa ta powinna umożliwiać przechowywanie takich wartości jak: imię, nazwisko, data urodzenia, płeć oraz cechować się zachowaniami w postaci zaimplementowanej metodzie zwracającej informację o osobie. Następnie zdefiniuj klasy dziedziczące po klasie Osoba:

1. Student – klasa powinna zawierać następujące składowe:
   a) pola
      - - nr indeksu
      - - typ studiów
      - - kierunek
      - - rok studiów
   b) metody
      - - zwracającą informację o studencie (należy przesłonić metodę odziedziczoną po klasie osoba). Można w tym celu wykorzystać metodę **toString()**.
2. Wykładowca – klasę tę należy zaprojektować samodzielnie (poprzez analogię do klasy Student) wykazując się pomysłowością i inwencją twórczą.

Osoba.java

```
public class Osoba {

    private String imie;
    private String nazwisko;
    private String dataUrodzenia; // format: DD-MM-RRRR
    private String plec;          // "M" lub "K"

    public Osoba(String imie, String nazwisko, String dataUrodzenia, String
plec) {
        this.imie = imie;
        this.nazwisko = nazwisko;
        this.dataUrodzenia = dataUrodzenia;
        this.plec = plec;
    }
```

```java
    public String getImie()           { return imie; }
    public String getNazwisko()       { return nazwisko; }
    public String getDataUrodzenia() { return dataUrodzenia; }
    public String getPlec()           { return plec; }

    /**
     * Metoda zwracająca informację o osobie.
     * Zostanie przesłonięta (@Override) w klasach pochodnych.
     */
    public String getInfo() {
        return "Imie: " + imie
             + ", Nazwisko: " + nazwisko
             + ", Data urodzenia: " + dataUrodzenia
             + ", Plec: " + plec;
    }

    @Override
    public String toString() {
        return getInfo();
    }
}
```

Student.java

```java
public class Student extends Osoba {

    private String nrIndeksu;
    private String typStudiow;
    private String kierunek;
    private int    rokStudiow;

    public Student(String imie, String nazwisko, String dataUrodzenia,
String plec,
                   String nrIndeksu, String typStudiow, String kierunek,
int rokStudiow) {
        // Wywolanie konstruktora klasy nadrzednej
        super(imie, nazwisko, dataUrodzenia, plec);
        this.nrIndeksu  = nrIndeksu;
        this.typStudiow = typStudiow;
        this.kierunek   = kierunek;
        this.rokStudiow = rokStudiow;
    }

    public String getNrIndeksu()  { return nrIndeksu; }
    public String getTypStudiow() { return typStudiow; }
    public String getKierunek()   { return kierunek; }
    public int    getRokStudiow() { return rokStudiow; }

    /** Przesloniecie getInfo() z klasy Osoba. */
    @Override
    public String getInfo() {
        return super.getInfo()
             + " | Nr indeksu: " + nrIndeksu
             + ", Typ studiow: " + typStudiow
             + ", Kierunek: "    + kierunek
             + ", Rok studiow: " + rokStudiow;
    }
```

```
    @Override
    public String toString() {
        return "[STUDENT] " + getInfo();
    }
}
```

Wykladowca.java

```
public class Wykladowca extends Osoba {

    private String nrPracowniczy;
    private String tytulNaukowy;
    private String katedra;
    private String specjalizacja;
    private int    stazPracy;

    public Wykladowca(String imie, String nazwisko, String dataUrodzenia,
String plec,
                      String nrPracowniczy, String tytulNaukowy,
                      String katedra, String specjalizacja, int stazPracy)
{
        super(imie, nazwisko, dataUrodzenia, plec);
        this.nrPracowniczy = nrPracowniczy;
        this.tytulNaukowy  = tytulNaukowy;
        this.katedra        = katedra;
        this.specjalizacja  = specjalizacja;
        this.stazPracy      = stazPracy;
    }

    public String getNrPracowniczy()  { return nrPracowniczy; }
    public String getTytulNaukowy()   { return tytulNaukowy; }
    public String getKatedra()        { return katedra; }
    public String getSpecjalizacja()  { return specjalizacja; }
    public int    getStazPracy()      { return stazPracy; }

    /** Przesloniecie getInfo() z klasy Osoba. */
    @Override
    public String getInfo() {
        return super.getInfo()
             + " | Nr pracowniczy: " + nrPracowniczy
             + ", Tytul: "           + tytulNaukowy
             + ", Katedra: "         + katedra
             + ", Specjalizacja: "   + specjalizacja
             + ", Staz pracy: "      + stazPracy + " lat";
    }

    @Override
    public String toString() {
        return "[WYKLADOWCA] " + getInfo();
    }
}
```

MainZadanie1.java

```
public class MainZadanie1 {

    public static void main(String[] args) {
        Osoba osoba = new Osoba("Jan", "Kowalski", "01-01-1990", "M");
```

```java
        Student student = new Student(
            "Dominik", "Delimata", "10-11-2004", "M",
            "141319", "stacjonarne", "Informatyka", 2
        );

        Wykladowca wykladowca = new Wykladowca(
            "Piotr", "Wisniewski", "20-06-1975", "M",
            "w001", "dr hab.",
            "Katedra Informatyki", "Programowanie obiektowe", 15
        );

        System.out.println("--- Obiekt Osoba ---");
        System.out.println(osoba);

        System.out.println();
        System.out.println("--- Obiekt Student ---");
        System.out.println(student);

        System.out.println();
        System.out.println("--- Obiekt Wykladowca ---");
        System.out.println(wykladowca);

        // Polimorfizm: tablica nadklasy przechowuje obiekty roznych
podklas
        System.out.println();
        System.out.println("--- Polimorfizm: tablica Osoba[] ---");
        Osoba[] osoby = { osoba, student, wykladowca };
        for (Osoba o : osoby) {
            // Wywoływana jest metoda toString() odpowiednia dla
rzeczywistego typu
            System.out.println(o);
        }

        System.out.println();
        System.out.println("--- Sprawdzenie instanceof ---");
        for (Osoba o : osoby) {
            System.out.println(o.getNazwisko()
                + " -> Student? "    + (o instanceof Student)
                + " | Wykladowca? " + (o instanceof Wykladowca));
        }
    }
}
```

```
C:\Users\Admin\.jdks\openjdk-25.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=52556" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -clas
--- Obiekt Osoba ---
Imie: Jan, Nazwisko: Kowalski, Data urodzenia: 01-01-1990, Plec: M

--- Obiekt Student ---
[STUDENT] Imie: Dominik, Nazwisko: Delimata, Data urodzenia: 10-11-2004, Plec: M | Nr indeksu: 141319, Typ studiow: stacjonarne, Kierunek: Informatyka, Rok studiow: 2

--- Obiekt Wykladowca ---
[WYKLADOWCA] Imie: Piotr, Nazwisko: Wisniewski, Data urodzenia: 20-06-1975, Plec: M | Nr pracowniczy: w001, Tytul: dr hab., Katedra: Katedra Informatyki, Specjalizacja: Programowanie obiektowe, Staz pracy: 15 lat

--- Polimorfizm: tablica Osoba[] ---
Imie: Jan, Nazwisko: Kowalski, Data urodzenia: 01-01-1990, Plec: M
[STUDENT] Imie: Dominik, Nazwisko: Delimata, Data urodzenia: 10-11-2004, Plec: M | Nr indeksu: 141319, Typ studiow: stacjonarne, Kierunek: Informatyka, Rok studiow: 2
[WYKLADOWCA] Imie: Piotr, Nazwisko: Wisniewski, Data urodzenia: 20-06-1975, Plec: M | Nr pracowniczy: w001, Tytul: dr hab., Katedra: Katedra Informatyki, Specjalizacja: Programowanie obiektowe, Staz pracy: 15 lat

--- Sprawdzenie instanceof ---
Kowalski -> Student? false | Wykladowca? false
Delimata -> Student? true | Wykladowca? false
Wisniewski -> Student? false | Wykladowca? true
```

Dziedziczenie eleminuje powielaniu kodu
pola imie/nazwisko/plec/dataUrodzenia żyją **raz** w Osoba i automatycznie pojawiają się w Student i Wykladowca. Przesłonięcie toString() przez @Override daje efekt polimorfizmu: wypisując obiekty z tablicy Osoba[] każdy obiekt "opisuje się sam" zgodnie z rzeczywistym typem. Słowo super(...) w konstruktorze podklasy przekazuje dane do nadklasy bez ich powielania.

Zadanie 2. Stwórz klasę Punkt2D, która przechowuje informacje opisujące punkt na płaszczyźnie dwuwymiarowej (współrzędne x oraz y). Zawiera ona dwa konstruktory: bezparametrowy ustawiający pola na wartość 0, oraz przyjmujący dwa argumenty i ustawiający pola obiektu zgodnie z podanymi parametrami. Utwórz metodę losującą współrzędne punktu na płaszczyźnie. Współrzędne mają być losowane w zakresie od -10 do 10. Przesłoń metodę toString() tak aby wyświetlała informacje o współrzędnych punktu w płaszczyźnie.

Napisz klasę Punkt3D reprezentującą punkt w trójwymiarze. Klasa ma powstać na bazie klasy Punkt2D z dodatkowym polem z opisującym trzeci wymiar. Przesłoń metodę losującą współrzędne punku tak aby losowała również trzeci wymiar (z). Przesłoń metodę toString() tak aby wyświetlała informacje o współrzędnych punktu w przestrzeni.

W klasie testowej utwórz obiekty obu klas i przetestuj działanie. Następnie utwórz dwie tablice 100-elementowe, jedna dla klasy punkt2D o nazwie array2D, a druga dla klasy Punkt3D o nazwie array3D. W obydwu tablicach wylosuj dla wszystkich elementów punkty przy użyciu utworzonych wcześniej metod. Sprawdź czy w tablicy array3D i array2D istnieją elementy mające wspólne składowe (x,y) tj. istnieją punkty punkt2D(x1,y1) i punkt3D(x2,y2,z2), takie że x1=x2 i y1=y2. Jeśli takie pary istnieją wypisz je na ekranie (wykorzystaj metodę toString().

Punkt2D.java

```
import java.util.Random;

public class Punkt2D {

    // protected - dostepne w podklasie Punkt3D bezposrednio
    protected int x;
    protected int y;
    private static final Random RAND = new Random();

    /** Konstruktor bezparametrowy - ustawia obie wspolrzedne na 0. */
    public Punkt2D() {
        this.x = 0;
```

```java
        this.y = 0;
    }

    /** Konstruktor parametryczny. */
    public Punkt2D(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }
    public int getY() { return y; }

    /**
     * Losuje wspolrzedne x i y w zakresie od -10 do 10 (wlacznie).
     * nextInt(21) -> [0..20], odjecie 10 -> [-10..10]
     */
    public void losujWspolrzedne() {
        this.x = RAND.nextInt(21) - 10;
        this.y = RAND.nextInt(21) - 10;
    }

    @Override
    public String toString() {
        return "Punkt2D(" + x + ", " + y + ")";
    }
}
```

Punkt3D.java

```java
import java.util.Random;

public class Punkt3D extends Punkt2D {

    private int z;
    private static final Random RAND = new Random();

    /** Konstruktor bezparametrowy - x=0, y=0, z=0. */
    public Punkt3D() {
        super();
        this.z = 0;
    }

    /** Konstruktor parametryczny. */
    public Punkt3D(int x, int y, int z) {
        super(x, y);
        this.z = z;
    }

    public int getZ() { return z; }

    /**
     * Przesloniecie metody losujacej.
     * Wywoluje super.losujWspolrzedne() (losuje x i y z nadklasy),
     * nastepnie losuje niezaleznie wartosc z.
     */
    @Override
    public void losujWspolrzedne() {
        super.losujWspolrzedne();        // losowanie x i y przez nadklase
        this.z = RAND.nextInt(21) - 10; // losowanie z
    }
```

```
    @Override
    public String toString() {
        return "Punkt3D(" + x + ", " + y + ", " + z + ")";
    }
}
```

MainZadanie2.java

```
/**
 * Klasa testowa dla Zadania 2.
 *
 * Kroki:
 *   1. Tworzy obiekty Punkt2D i Punkt3D, testuje konstruktory i
toString().
 *   2. Tworzy dwie tablice 100-elementowe: array2D i array3D.
 *   3. Losuje wspolrzedne dla wszystkich elementow.
 *   4. Szuka par (p2D, p3D) takich, ze p2D.x == p3D.x ORAZ p2D.y == p3D.y.
 *   5. Wypisuje znalezione pary na ekranie (metoda toString()).
 */
public class MainZadanie2 {

    public static void main(String[] args) {

        System.out.println("=== ZADANIE 2: Dziedziczenie - Punkt2D i
Punkt3D ===");
        System.out.println();

        // 1. Test pojedynczych obiektow
        Punkt2D p1 = new Punkt2D();          // konstruktor bezparametrowy
        Punkt2D p2 = new Punkt2D(3, -5);     // konstruktor parametryczny
        Punkt3D p3 = new Punkt3D();          // bezparametrowy
        Punkt3D p4 = new Punkt3D(1, 2, 3); // parametryczny

        System.out.println("Punkt2D (bezparametrowy): " + p1);
        System.out.println("Punkt2D (parametryczny):  " + p2);
        System.out.println("Punkt3D (bezparametrowy): " + p3);
        System.out.println("Punkt3D (parametryczny):  " + p4);

        p1.losujWspolrzedne();
        p3.losujWspolrzedne();
        System.out.println();
        System.out.println("Po losowaniu:");
        System.out.println("p1 losowy: " + p1);
        System.out.println("p3 losowy: " + p3);

        // 2. Tablice 100-elementowe
        Punkt2D[] array2D = new Punkt2D[100];
        Punkt3D[] array3D = new Punkt3D[100];

        // 3. Losowanie wspolrzednych
        for (int i = 0; i < 100; i++) {
            array2D[i] = new Punkt2D();
            array2D[i].losujWspolrzedne();
            array3D[i] = new Punkt3D();
            array3D[i].losujWspolrzedne();
        }

        // 4 & 5. Wyszukiwanie par ze wspolnymi (x, y)
        System.out.println();
```

```
        System.out.println("--- Pary punktow o wspolnych skladowych (x, y)
---");
        int licznikPar = 0;

        for (Punkt2D pkt2 : array2D) {
            for (Punkt3D pkt3 : array3D) {
                if (pkt2.getX() == pkt3.getX() && pkt2.getY() ==
pkt3.getY()) {
                    System.out.println("  Wspolne: " + pkt2 + "  <-->  " +
pkt3);
                    licznikPar++;
                }
            }
        }

        if (licznikPar == 0) {
            System.out.println("  Nie znaleziono par o wspolnych
wspolrzednych (x, y).");
        } else {
            System.out.println();
            System.out.println("Laczna liczba znalezionych par: " +
licznikPar);
        }

        // Bonus: polimorfizm - tablica Punkt2D[] przechowuje Punkt3D
        System.out.println();
        System.out.println("--- Polimorfizm: Punkt2D[] przechowujacy
Punkt3D ---");
        Punkt2D[] mieszana = new Punkt2D[4];
        mieszana[0] = new Punkt2D(1, 1);
        mieszana[1] = new Punkt3D(1, 1, 5);   // Punkt3D traktowany jako
Punkt2D
        mieszana[2] = new Punkt2D(2, 3);
        mieszana[3] = new Punkt3D(2, 3, -7);

        for (Punkt2D p : mieszana) {
            // toString() wywoływane polimorficznie
            System.out.println("  " + p);
        }
    }
}
```

```
Punkt2D (bezparametrowy): Punkt2D(0, 0)
Punkt2D (parametryczny):  Punkt2D(3, -5)
Punkt3D (bezparametrowy): Punkt3D(0, 0, 0)
Punkt3D (parametryczny):  Punkt3D(1, 2, 3)

Po losowaniu:
p1 losowy: Punkt2D(6, -9)
p3 losowy: Punkt3D(-7, -5, -1)

--- Pary punktow o wspolnych skladowych (x, y) ---
  Wspolne: Punkt2D(-2, 4)  <-->  Punkt3D(-2, 4, -5)
  Wspolne: Punkt2D(6, 2)  <-->  Punkt3D(6, 2, 10)
  Wspolne: Punkt2D(2, -7)  <-->  Punkt3D(2, -7, 2)
  Wspolne: Punkt2D(2, -7)  <-->  Punkt3D(2, -7, 2)
  Wspolne: Punkt2D(6, 9)  <-->  Punkt3D(6, 9, 4)
  Wspolne: Punkt2D(1, 6)  <-->  Punkt3D(1, 6, -1)
  Wspolne: Punkt2D(6, 2)  <-->  Punkt3D(6, 2, 10)
  Wspolne: Punkt2D(4, -8)  <-->  Punkt3D(4, -8, 6)
  Wspolne: Punkt2D(4, -8)  <-->  Punkt3D(4, -8, -5)
  Wspolne: Punkt2D(5, 6)  <-->  Punkt3D(5, 6, 9)
  Wspolne: Punkt2D(1, -1)  <-->  Punkt3D(1, -1, -3)
  Wspolne: Punkt2D(5, 1)  <-->  Punkt3D(5, 1, -8)
  Wspolne: Punkt2D(9, 2)  <-->  Punkt3D(9, 2, -7)
  Wspolne: Punkt2D(1, -8)  <-->  Punkt3D(1, -8, -7)
  Wspolne: Punkt2D(0, 9)  <-->  Punkt3D(0, 9, -1)
  Wspolne: Punkt2D(0, 9)  <-->  Punkt3D(0, 9, 7)
  Wspolne: Punkt2D(-6, -7)  <-->  Punkt3D(-6, -7, -2)
  Wspolne: Punkt2D(7, -3)  <-->  Punkt3D(7, -3, 9)
  Wspolne: Punkt2D(9, 10)  <-->  Punkt3D(9, 10, 5)
  Wspolne: Punkt2D(-1, 3)  <-->  Punkt3D(-1, 3, -5)
  Wspolne: Punkt2D(1, -8)  <-->  Punkt3D(1, -8, -7)
  Wspolne: Punkt2D(5, 7)  <-->  Punkt3D(5, 7, 0)
  Wspolne: Punkt2D(-10, 0)  <-->  Punkt3D(-10, 0, -2)

Laczna liczba znalezionych par: 23

--- Polimorfizm: Punkt2D[] przechowujacy Punkt3D ---
  Punkt2D(1, 1)
  Punkt3D(1, 1, 5)
  Punkt2D(2, 3)
  Punkt3D(2, 3, -7)
```

Użycie protected zamiast private dla x i y pozwala Punkt3D odczytywać je bezpośrednio w toString() bez getterów. Wywołanie super.losujWspolrzedne() w Punkt3D jest przykładem zasady **DRY** – logika losowania x i y istnieje w jednym miejscu. Szukanie par działa w czasie $O(n^2)$ = 10 000 porównań; statystycznie przy dziedzinie [-10,10] (21 wartości) oczekiwane ~23 pary na 10 000 kombinacji (1/21)2≈0,23%(1/21)2≈0,23%.